

B B M

Beauty Business Manager

Software Requirements Specification

Summary Draft — v0.2

PREPARED FOR TSA Developers — Product Planning

DERIVED FROM Beauty Business Manager: System Design & Implementation Guide (Draft v1.0)

PURPOSE Consolidate the design guide into a requirements baseline before implementation

MARKET Tanzanian cosmetics & beauty retailers (TZS)

STATUS Draft — for internal review

1. Introduction

1.1 Purpose

This document restates the Beauty Business Manager (BBM) design guide as a requirements baseline: what the system must do, who it must do it for, and the constraints it must respect. It exists so the team can implement against agreed requirements rather than re-reading the full design guide narrative at each phase. The design guide remains the authoritative source for schema detail, API shapes, and phase sequencing — this SRS is a summary and requirements framing of that content, not a replacement.

1.2 Scope

BBM is a multi-tenant SaaS platform for cosmetics, skincare, and beauty retailers in Tanzania. It covers authentication and role-based access, per-tenant product catalog and inventory (including batch/expiry tracking), the sale transaction and its financial consequences (COGS, gross/net profit), purchasing, expense tracking, reporting, staff management, and a notification abstraction (in-app first; SMS/WhatsApp/email as later, pluggable channels).

1.3 Confirmed stack

Frontend: React (Vite), TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form, Zod, Recharts. **Backend:** Python, Django, Django REST Framework, DRF Serializers, Django ORM, Django migrations. **Data/infra:** PostgreSQL 15+ with UUID primary keys, Redis for caching, Celery (or RQ) for background jobs, Docker Compose, Nginx reverse proxy. **Auth:** JWT access + rotating refresh tokens (djangorestframework-simplejwt), Argon2 password hashing. This supersedes an earlier Next.js/FastAPI stack note recorded for this project — the React/Vite + DRF combination above is the one to build against. As noted in the design guide (Section 3), consider deferring Celery to Phase 4 and running scheduled jobs via a cron-triggered management command until then, to remove a class of local-dev friction early on.

1.4 Definitions

Term	Meaning
Tenant	A single beauty retail business using BBM; owns its own users, products, sales, and data.
COGS	Cost of Goods Sold — the buying-price cost of items sold, snapshotted at sale time.
Stock movement	An append-only ledger row recording any change to product stock quantity.
RBAC	Role-Based Access Control — permissions granted through roles (Owner, Admin) rather than per-user.

2. Overall Description

2.1 Product perspective

BBM is a new, standalone SaaS product — not an extension of an existing TSA system. It is built and delivered in five phases (Phase 0 through Phase 5), each ending in something a real business could use, rather than as one monolithic release. See Appendix A for the phase roadmap.

2.2 User classes

Role	Description
Owner	Full access: users, roles, billing/settings, all financial data, delete operations, audit log access.

Role	Description
Admin	Day-to-day operations: products, inventory, sales, purchases, expenses, staff creation — no billing, deletion, or audit-log purge.
(Future) Cashier / Accountant	Not built yet; the permissions model is designed so these can be added without touching business logic.

2.3 Operating environment

Docker Compose-orchestrated services (frontend, backend, PostgreSQL, Redis, Nginx reverse proxy). Mobile-first responsive frontend; desktop sidebar navigation, mobile bottom navigation/drawer. Currency is TZS; the target market operates heavily over WhatsApp/Instagram, which should inform UX even though those channels are not wired until Phase 5.

2.4 Design & implementation constraints

- Row-level, shared-schema multi-tenancy: every tenant-owned table carries **business_id**; scoping is enforced centrally at the query layer, never left to individual views.
- **business_id** is always derived from the authenticated JWT — never accepted from a client-supplied body or query parameter.
- Every stock change is a ledger row (**stock_movements**); current stock is always a derived sum, never a directly-edited field.
- **sale_items** and **purchase_items** snapshot unit price/cost at transaction time — product price changes must never rewrite historical profit.
- Unconfigured notification channels (SMS/WhatsApp/email) must fail visibly, never silently pretend to succeed.

3. Functional Requirements

Grouped by module and tagged with priority and the phase that delivers them. Endpoint-level detail is in the design guide, Section 8; this table is the requirement statement, not the API contract.

3.1 Identity, Tenancy & Access

ID	Requirement	Priority	Phase
FR-1	System shall create a User, Business, Owner role, and default Settings row in a single atomic transaction on registration.	Critical	1
FR-2	System shall issue short-lived JWT access tokens and rotating refresh tokens; each refresh invalidates the prior refresh token.	Critical	1
FR-3	System shall scope every tenant-owned query by business_id derived from the JWT, enforced at a single central layer.	Critical	1
FR-4	System shall support password reset via a time-limited, single-use token, without revealing account existence.	High	1
FR-5	System shall enforce granular, table-driven permissions (not hardcoded role checks) for Owner vs Admin actions.	High	1

3.2 Catalog & Inventory

ID	Requirement	Priority	Phase
FR-6	System shall support product CRUD with category, brand, supplier, batch, and expiry fields, scoped per tenant.	High	2
FR-7	System shall record every stock change (sale, purchase, damage, adjustment, transfer, expiry) as a stock_movements row.	Critical	2
FR-8	System shall derive current stock as a sum over stock_movements, never accept a direct stock-quantity edit.	Critical	2
FR-9	System shall surface low-stock and expiry (7/30/60-day, expired) views from live queries, filterable by category/brand/supplier.	High	2
FR-10	System shall validate uploaded product images by file type and size before storage.	Medium	2

3.3 Sales, Purchases & Expenses

ID	Requirement	Priority	Phase
FR-11	System shall process a sale as one atomic transaction: validate tenant + stock, create sale + items, decrement stock, write stock movement(s), compute COGS/profit, record payment.	Critical	3
FR-12	System shall roll back the entire sale transaction, including any stock decrement, if any step from item-creation onward fails.	Critical	3
FR-13	System shall block sales exceeding available stock by default, with an explicit, audited per-business override to allow overselling.	High	3
FR-14	System shall increase inventory and write a PURCHASE stock movement when a purchase is completed, in the same transaction.	High	3

ID	Requirement	Priority	Phase
FR-15	System shall support categorised expense entry with optional validated receipt attachment.	Medium	3

3.4 Reporting, Dashboard & Notifications

ID	Requirement	Priority	Phase
FR-16	System shall compute gross profit as revenue minus COGS (using the snapshotted buying price), and net profit as gross profit minus period expenses.	Critical	3
FR-17	System shall present a dashboard (today's sales, profit, expenses, transaction count, low-stock/expiry counts) computed from live queries, date-range filterable.	High	3
FR-18	System shall provide daily/weekly/monthly/annual sales, profit, expense, stock-valuation and expiry reports with CSV export.	Medium	4
FR-19	System shall provide an in-app notification centre; SMS/WhatsApp/email shall exist as configured-but-inactive interfaces until a provider exists, and must fail visibly if used unconfigured.	Medium	4
FR-20	System shall log sensitive actions (role changes, deletions, settings changes, exports) to an audit log accessible only to Owner.	High	4

3.5 Deferred (Phase 5)

- FR-21 (Medium) — Loyalty points and accounts.
- FR-22 (Medium) — Two-factor authentication and print-formatted receipts.
- FR-23 (Medium) — Real SMS/WhatsApp provider integration once credentials exist.
- FR-24 (Low) — Swahili localisation layer.

4. Non-Functional Requirements

4.1 Security

ID	Requirement	Priority
NFR-1	Passwords hashed with Argon2 (or bcrypt); never plaintext or reversible encryption.	Critical
NFR-2	All input validated on both frontend (Zod) and backend (server-side schema/serializer validation) — frontend validation is never trusted alone.	Critical
NFR-3	Rate limiting on authentication endpoints and public-facing forms before real customers are charged.	High
NFR-4	CORS locked to known frontend origins; secure headers (HSTS, X-Content-Type-Options) applied at the reverse proxy.	High
NFR-5	File uploads restricted by type allow-list and size cap, stored outside the web root or in object storage.	High
NFR-6	Automated database backups exist and have been restored successfully at least once in a test environment.	Critical

4.2 Multi-tenancy & data integrity

ID	Requirement	Priority
NFR-7	A second tenant must never be able to read, update, or delete another tenant's record via any endpoint, including by guessing a valid UUID.	Critical
NFR-8	Composite indexes on (business_id, id) for every tenant table, and (business_id, created_at) for date-ranged tables (sales, expenses, stock movements).	High
NFR-9	Total stock-in minus stock-out must reconcile exactly against the stock_movements ledger for any product at any time.	Critical

4.3 Reliability & maintainability

ID	Requirement	Priority
NFR-10	Every list view ships with loading, empty, and error states plus pagination and search from its first version.	Medium
NFR-11	CI runs lint and tests on every push from Phase 0 onward.	Medium
NFR-12	README and docs/ cover setup, architecture, API, database, deployment, and security posture per phase.	Medium

5. Data Requirements (Core Entities)

UUID primary keys and created_at/updated_at timestamps throughout; every tenant-owned table carries business_id. Full column-level schema is in the design guide, Section 6 — this is the entity summary.

Entity	Tenant-scoped	Purpose
users	No*	Login identity; linked to businesses via user_roles.
businesses	—	One row per tenant; profile fields (name, logo, TIN, currency).

Entity	Tenant-scoped	Purpose
roles / permissions / user_roles	Yes	RBAC join tables scoping a role to a specific business.
categories / brands / suppliers	Yes	Per-tenant reference/taxonomy data.
products / product_batches	Yes	Catalog items and their batch/expiry records.
stock_movements	Yes	Append-only ledger of every stock quantity change.
sales / sale_items	Yes	Sale header and line items; items snapshot price and cost.
purchases / purchase_items	Yes	Purchase header and line items; drives stock movements on completion.
expenses / expense_categories	Yes	Operating cost tracking feeding net profit.
loyalty_accounts / loyalty_transactions	Yes	Phase 5 — schema reserved now, built later.
notifications	Yes	In-app notification records; channel field reserved for email/SMS/WhatsApp.
settings	Yes	Per-business configuration: thresholds, tax, currency, loyalty toggle.
audit_logs	Yes	Who did what, when — required before charging real customers.

** users is not tenant-scoped directly; the association is via user_roles, since one login could in principle belong to more than one business.*

6. External Interface Requirements

6.1 API

REST, versioned under /api/, documented via an auto-generated OpenAPI schema. Representative domains: Auth, Dashboard, Products, Inventory, Sales, Purchases, Expenses, Reports, Staff & Roles, Notifications, Settings — each following the same list/detail/create/update/delete pattern. Full endpoint map: design guide Section 8.

6.2 Third-party / deferred interfaces

- SMTP for verification and password-reset email (stub-to-console acceptable pre-Phase 4).
- SMS provider — interface reserved, no live credentials until a provider is chosen (Phase 5).
- WhatsApp Business API — interface reserved, not wired until Phase 5.
- Object storage endpoint for product images and receipt attachments.

7. Constraints & Assumptions

- Single full-time developer is the baseline planning assumption; phase durations halve with a second engineer split across backend/frontend.
- No real business uses the system until Phase 3 delivers a working sale transaction — validation of UX and workflow assumptions is deliberately deferred, not skipped.
- Currency is TZS throughout; no multi-currency requirement at this stage.
- Target users operate substantially over WhatsApp/Instagram today — this should inform UX decisions even though those channels are not integrated until Phase 5.
- Tanzania's Personal Data Protection Act applies to how tenant and customer data is stored and handled (noted at the TSA Developers program level; not itemised as line-item requirements in the design guide).

Appendix A — Phase Roadmap (Reference)

Phase	Goal	Est. duration
0 — Foundations	A running skeleton with nothing fake in it.	~1 week
1 — Identity & Tenant Core	Owner can register, log in, land on an empty but real dashboard.	~2 weeks
2 — Catalog & Inventory	Owner can manage real products and see real stock levels.	~2–3 weeks
3 — The Money Path	A sale can be recorded end-to-end with correct numbers.	~2–3 weeks
4 — Reporting, Roles & Hardening	Presentable to a second business; safe to charge for.	~2–3 weeks
5 — Nice-to-Have Layer	Loyalty, 2FA, real SMS/WhatsApp, localisation.	As needed

This SRS summarises and re-frames the Beauty Business Manager System Design & Implementation Guide (Draft v1.0) as requirements, built against the confirmed React (Vite) + Django REST Framework stack in Section 1.3.